ChatGPT

Scaling Strategies & Forecasting Capability Frontiers

Introduction

Scaling up AI models has been a driving force behind recent breakthroughs in machine learning, especially with large language models (LLMs). **Scaling strategies** refer to how we increase model size, training data, and compute in order to improve performance. By studying empirical *scaling laws*, researchers have uncovered predictable relationships between these factors and model capability [1]. These insights enable **compute-optimal planning** – figuring out the best way to allocate a fixed compute budget across model parameters, number of training tokens, and training steps for maximum performance [2]. At the same time, as models approach higher capability, we face questions of **data quality** (are we running out of high-quality data to feed ever-larger models?), **reliability of reasoning** (do bigger models reason more robustly or do new failure modes emerge?), and ultimately **forecasting "AGI" timelines** (predicting when AI might reach human-level general intelligence). This study guide will cover these topics in detail, drawing on key papers: Kaplan *et al.* (2020) on early scaling laws, Hoffmann *et al.* (2022) on *compute-optimal* training (the *Chinchilla* study), Sagawa *et al.* (2023) on capability forecasting (scaling perspectives), and Bubeck *et al.* (2023) on emergent abilities in GPT-4. We will also discuss how to formulate a **group scaling-plan proposal** – effectively applying scaling laws in practice – in a case-study style. Throughout, we'll use real scaling curves and examples from the literature to illustrate concepts.

Compute-Optimal Scaling: Parameters ↔ Tokens ↔ Steps

Kaplan et al. (2020): Power-Law Scaling and Optimal Compute Allocation

In 2020, Kaplan *et al.* published a seminal work on *scaling laws for neural language models* [3]. They trained language models across **seven orders of magnitude** in scale and observed that the *test loss* (cross-entropy) follows a clear **power-law decline** as functions of three key factors: **model size (N)**, **dataset size (D)**, and **training compute (C)** [2] [1]. Figure 1 below (from Kaplan *et al.*) shows the core result: each plot is linear on a log-log scale, indicating a power-law relationship. Performance improves (loss decreases) smoothly as we increase compute, data, or model parameters, until eventually hitting an irreducible loss floor (due to the intrinsic entropy of natural language) [4] [5].

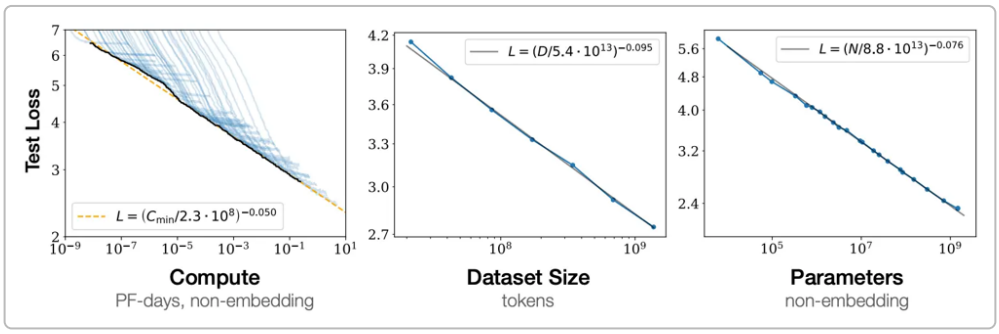


Figure 1: Scaling laws from Kaplan et al. (2020) [4] [5] . Test loss follows a power law as model parameters (right), dataset size in tokens (center), or compute in petaflop-days (left) increase. Doubling model size, for example, reduces loss by a constant factor (~5% reduction per doubling) [5] .

A remarkable finding was that these trends held over vast scales without saturating (within the tested range). The authors summarized the relationships in simple power-law forms. For instance, they estimated:

- **Model size law:** $L(N) \propto N^{-0.076}$ – doubling the number of non-embedding parameters *N* reduces loss by about 5% [5] .
- **Data size law:** $L(D) \propto D^{-0.095}$ – doubling the training tokens reduces loss ~6.4% [5] .
- **Compute law:** $L(C) \propto C^{-0.050}$ – doubling total compute (FLOPs) yields ~3.4% less loss [5] .

These exponents (0.076, 0.095, 0.050) quantify *diminishing returns*: more data helps more than more parameters (per doubling), and both help more than longer training on the same data. Importantly, Kaplan *et al.* also fit *multivariate* scaling laws to capture interactions. They proposed a combined loss function of model and data:

$$L(N, D) = \left[(N/N_c)^{\alpha_N} + (D/D_c)^{\alpha_D} \right]^{-1/\alpha_D},$$

with $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.103$ (and N_c, D_c constants) [6] [7] . This equation implies an **optimal trade-off**: given a fixed compute budget, there is a best way to split it between training a larger model vs. training on more data. Kaplan *et al.* used this to determine the **compute-optimal allocation** of model size and training duration.

The compute-optimal frontier: Kaplan *et al.* found that *larger models are significantly more sample-efficient* – they can reach a given loss with much fewer training examples than smaller models [8] . Therefore, with a fixed compute budget, the optimal strategy is to use a *very large model* and train it on a **relatively modest amount of data, stopping well before full convergence** [8] . In other words, it's better to train a bigger model for fewer steps than a smaller model to completion. This was a somewhat counterintuitive result at the time. Mathematically, for a 10× increase in available compute, Kaplan's law suggested scaling *model parameters* about 5.5× and *dataset size* only about 1.8× [9] . In effect, *model size* was emphasized more than *training duration* or data volume under their optimal scaling rule. This guidance – *"scale model width/depth aggressively, and don't necessarily use all your data"* – influenced the design of models like **GPT-3**, which had 175B parameters but was trained on ~300 billion tokens (stopping short of using any more data to avoid over-training given the budget).

However, an important caveat emerged: Kaplan's analysis didn't account for an absolute lower bound on loss (the "irreducible" loss from data entropy) [10] . Their power laws had no explicit floor, which means they could overestimate the benefits of scaling when loss approaches that floor. By 2022, as models grew much larger (multi-hundred-billion parameters), researchers began to notice discrepancies from the 2020 predictions. This brings us to the *next* major development in scaling laws.

Hoffmann et al. (2022): Revisiting Compute-Optimal Training (the Chinchilla Paper)

Hoffmann *et al.* (2022) – often known by the nickname "*Chinchilla*" paper – revisited scaling laws using new data and larger models. They made a pivotal discovery: *"current large language models are significantly*

undertrained" given their size [11]. In recent years, researchers had been scaling up model sizes rapidly (GPT-2 → GPT-3 → Gopher (280B) → Megatron (530B), etc.) while keeping the amount of training data roughly constant (on the order of a few hundred billion tokens). Hoffmann *et al.* trained over 400 models ranging from 70M up to 16B parameters on anywhere from 5B to 500B tokens, systematically exploring how to best use a fixed compute budget [12]. Their finding was that **Kaplan's strategy was no longer optimal at these scales**. Instead of training very large models briefly, **it's better to use a somewhat smaller model and train it on many more tokens** – in fact, *model size and data size should be scaled in equal measure* as compute increases [13].

In quantitative terms, Hoffmann *et al.* found that for compute-optimal training:

- Model size $N \propto C^{0.50}$
- Training tokens $D \propto C^{0.50}$

approximately [14] [15]. This implies a **one-to-one scaling**: for each 2× increase in compute budget, use ~2× larger model and ~2× more data (i.e. train twice as long). The contrast with Kaplan's result (which had $N \propto C^{0.73}$, $D \propto C^{0.27}$ [9]) is stark – the new analysis shifts more emphasis onto **training duration (data)** than before. Figure 2 illustrates this updated optimal frontier: the red dotted line in the middle and right plots shows the ideal balance between model size and number of training FLOPs, which corresponds to a roughly constant ratio of tokens to parameters [14] [16]. In practice, Hoffmann *et al.* concluded that *for a given compute budget, you should roughly double the number of training tokens whenever you double the model size* [14] [15].

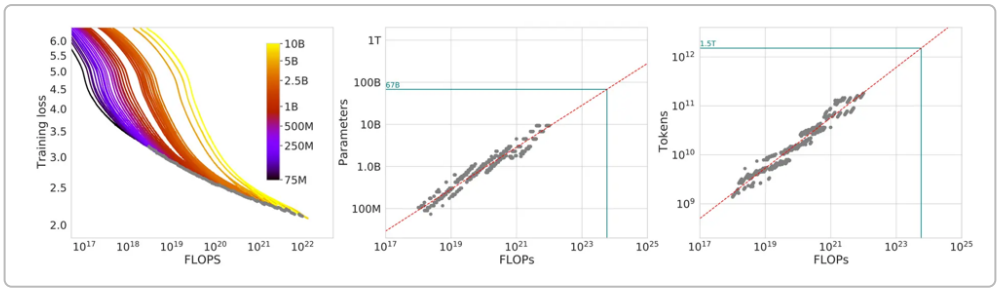


Figure 2: Visualization of the compute-optimal frontier from Hoffmann et al. (2022) [14] [16]. Left: Training loss vs. training FLOPs for various model sizes (color-coded). Middle: Optimal model size (parameters) for each FLOP budget lies on the red dotted line (slope ~0.5 in log-log). Right: Optimal training tokens (dataset size) for each FLOP budget also scale ~0.5 with FLOPs. The teal annotations (e.g. "1.5T" tokens) indicate Chinchilla's setting versus a larger model (Gopher) with fewer tokens.

Why the change? Hoffmann *et al.* identified a couple of reasons for the discrepancy from Kaplan's law [17] [18]:

- **Scale of models:** Kaplan's fits were based on models up to 1.5B parameters. By 2022, it was evident that beyond a certain size, models trained on fixed data were under-utilizing capacity. Hoffmann's study included models up to 16B (and they extrapolated to plan a 70B model), revealing new trends at larger scales [19].
- **Irreducible loss ("entropy"):** Kaplan's formulation lacked an explicit loss floor. Hoffmann *et al.* added an offset E in their parametric law to account for the minimum possible loss (e.g. predicting English

text has a baseline entropy). Including this term changes the optimal allocation, especially as models approach that floor [20] [21].

- **Training schedules:** Details like learning rate schedules can affect measured efficiency. Hoffmann *et al.* argued that some of Kaplan's models might have been slightly undertrained due to suboptimal hyperparameters for longer training [22]. They carefully tuned to ensure models were fully utilizing the allotted FLOPs.

To validate their new law, Hoffmann *et al.* made a bold test: they trained a model according to the *compute-optimal recipe* and checked its performance against larger but less-trained models. This model, named **Chinchilla**, had **70B parameters** and was trained on **1.4 trillion tokens** (4× more data than the 280B-parameter Gopher model used) [13] [23]. Notably, Chinchilla used the **same total compute** as Gopher – they essentially traded off 4× more training time for a 4× smaller model. The result: **Chinchilla uniformly and significantly outperformed Gopher (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B)** on a wide range of downstream tasks [23]. For example, on the MMLU academic exam benchmark, Chinchilla scored 67.5%, over 7% higher than Gopher despite Gopher having 4× more parameters [24]. This was a watershed moment – it proved that large models like GPT-3 had been trained in a suboptimal regime, and that **smaller models trained on more data can *beat* bigger models trained on less** given the same compute budget [13] [23]. In practical terms, this finding has massive implications: it means that many past large models could have been improved *not by adding more layers*, but by simply feeding them more data or training for more steps.

A convenient rule-of-thumb emerged from the Chinchilla analysis: *approximately 20 tokens per parameter is the data requirement for optimal training*. In other words, to fully train a model of size N (in parameters), one should plan for about 20×N training token consumption. (Chinchilla's 70B params × 20 = 1.4T tokens fits this heuristic.) GPT-3, by contrast, had ~175B params but only ~300B tokens (~1.7 tokens/param), which is far below the ~20:1 ratio – hence GPT-3 was undertrained and left a lot of potential performance on the table [11]. Today, most new LLMs adhere to this guidance; for instance, when OpenAI later trained GPT-4, or Meta trained LLaMA-2, it's believed they used much larger training sets to saturate those models' capacity (exact numbers are not always public, but following the Chinchilla scaling law has become standard).

Key takeaway: To maximize performance for a given compute budget, one should *balance* model size and dataset size. Early scaling laws suggested using extremely large models with fewer steps, but updated evidence shows that **training longer on more data (even with a smaller model) yields better results** [13] [23]. As a practitioner planning a project, this means if you have X GPU-days of compute, you shouldn't pour nearly all of it into making the model huge – you must also budget enough FLOPs to feed the model sufficient data. There is an optimal point (on the so-called "efficient frontier") where increasing the model further without more data will just lead to overfitting or under-utilization. Conversely, throwing excess data at too small a model leads to diminishing returns. The Chinchilla paper gives a clear prescription: allocate compute such that **model FLOPs ≈ data FLOPs** (roughly equal exponent contributions) in training.

Data-Quality Curves and the Limits of Scaling Data

If increasing data is so important, one might ask: why not just keep feeding our models more and more text? The catch is that **not all data is equal**. As we scale to using *trillions* of tokens, we inevitably have to include **lower-quality or redundant data** (e.g. content from the web that might be noisy, or repeats of the same information). This leads to the concept of **data-quality scaling curves** – how model performance improves as we add *additional data of varying quality*. Early scaling laws implicitly assumed we were adding

more of the same type of data; in reality, after a point, one runs out of *high-quality, diverse* text (such as Wikipedia articles, books, academic writings) and must include increasingly *long-tail or lower-quality* sources (forums, scraped web text, etc.). This can cause **sub-linear returns** or even **sub-scaling** behavior, where performance saturates sooner than the ideal power-law predicts.

Recent studies have observed exactly this: beyond a certain scale, *performance improvements decelerate* – the curve bends downward – when additional data doesn't provide much new information [25] [26]. Chen *et al.* (2025) identify *"high data density"* (i.e. a lot of repetitive or low-entropy content) as a key factor contributing to *sub-scaling* regimes [27]. In their analysis, training on extremely large but very redundant datasets yields diminishing returns: the model sees the same information over and over, so eventually each extra token contributes less to learning. Figure 3 (from their paper) illustrates this with loss curves: when using 5B tokens versus 500B tokens, the larger dataset shows a *curvature upward* – actual training loss stays higher than one would extrapolate from the small-data trend, indicating inefficiency for the largest models [28]. In plain terms, **throwing more data at the problem only helps if that data is bringing new signal rather than noise**.

Supporting evidence comes from multiple angles:
- Hernandez *et al.* (2022) and others found that simply repeating the same dataset (to get more tokens) leads to *diminishing returns in performance* [26]. Once the model has seen the data, going over it again yields less gain each epoch – better to have fresh data.
- Certain *task types* also exhibit sub-scaling. For example, **complex reasoning tasks** or tasks requiring composition show slower improvement – an effect some attribute to data quality (lack of targeted data for those skills) and the inherent difficulty of those tasks [29] [30]. In other words, scaling might not help as much on tasks that aren't well-represented or easily learnable from the available training distribution.

To combat this, researchers are exploring ways to improve data quality rather than quantity. One striking example is the **"Textbooks Are All You Need"** experiment by Gunasekar *et al.* (2023). They deliberately trained a smaller model on a *much smaller but higher-quality* dataset: a 1.3B parameter Transformer (named **phi-1**) was trained on only ~7 billion tokens of curated data – including **"textbook quality"** portions of the web and synthetically generated educational content [31]. Despite its tiny size and data budget, phi-1 achieved **50.6% pass@1 on the HumanEval coding benchmark** [31]. For context, reaching ~50% on HumanEval is a level of performance that previously required models like Codex with 12B or GPT-3.5 with tens of billions of parameters – in fact, no other 1B-scale model came close. Phi-1 essentially matched or beat models **1000× larger** on coding tasks by virtue of high-quality training data [31]. This demonstrates that *data quality can, to an extent, substitute for data quantity or model size*. Another way to put it: a smaller "brain" trained on *expert-curated knowledge* can outperform a bigger "brain" trained on the general internet.

Beyond hand-curation, other approaches to improving data quality include **filtering** (removing duplicates, errors, irrelevance), **up-sampling more informative subsets** (e.g. more coding data if you care about coding tasks), and using *model-in-the-loop data generation* (as phi-1 did by generating synthetic exercises via GPT-3.5 to supplement real data [31]). There is also research on *"inverse scaling" filters* [32] that try to predict which data will be most beneficial by using scaling laws in reverse (e.g. filtering out data that smaller models already learn easily, thus focusing on harder examples that big models still need).

It's worth noting that as of 2025, we are possibly approaching the limits of available high-quality text data for pretraining. Estimates suggest that web-scale corpora used for GPT-3 and others have already scraped a large fraction of the internet's easily accessible text. A 2023 analysis by Villalobos *et al.* even warned of a

potential **"data bottleneck"** around 2026-2027, where the demand for trillions of fresh tokens might outpace supply of new content (unless quality thresholds are lowered) [33]. This has two implications for scaling strategy:

- **Diminishing returns:** If future models are forced to include increasingly noisy data, we might see sub-scaling more often. Scaling laws might need adjustments (as in Chen et al. 2025's *"sub-optimal scaling law"* tailored for dense data [34] [35]). The power-law might bend sooner, indicating it's time to focus on data curation or new data modalities (like code, images, multilingual text, etc., which provide additional signal).
- **Synthetic data & active learning:** To keep scaling, researchers might generate training data (through simulation or human-in-the-loop) that is richer than random web text. The phi-1 result is a proof-of-concept: *a model taught with "textbook" content can do more with less*. Projects like DeepMind's *GopherCite* and OpenAI's use of human feedback can also be seen as efforts to improve effective data quality for better generalization.

In summary, **scaling the *quantity* of data alone is not enough** – one must also consider the *quality and diversity* of data. The ideal curve of "more data → better performance" will flatten out if the additional data doesn't teach the model new concepts. Modern scaling strategies therefore involve *data filtering, balancing, and augmentation* as first-class concerns. When planning a large training run, it's crucial to ask: *Have we exhausted the most informative data?* If so, simply scaling further might yield disappointing returns unless new sources or clever data-generation methods are brought in.

Reliability and Reasoning: Does Scale Yield Robust Intelligence?

One of the most exciting (and controversial) aspects of scaling up models has been the emergence of more advanced **reasoning capabilities**. With models like GPT-3, PaLM, and GPT-4, we've seen breakthroughs on tasks involving arithmetic, logic puzzles, coding, commonsense reasoning, and more – tasks that smaller models struggled with. This has led some to argue that *"reasoning emerges with scale"* as an intrinsic phenomenon [36] [37]. Indeed, Bubeck *et al.* (2023) famously subtitled their paper on GPT-4 *"Sparks of Artificial General Intelligence"*, because GPT-4 can solve novel and difficult problems across domains (math, coding, vision, law, etc.) **without specific training** beyond its general next-word prediction objective [38]. Impressively, in many of these tasks GPT-4's performance is *close to human-level* and vastly better than earlier models like GPT-3 or even ChatGPT (GPT-3.5) [38]. For example, GPT-4 can write code to solve competitive programming problems, prove basic theorems, explain jokes, and even generate functional web apps from a description – all capabilities that *did not clearly exist* in models half its size. The breadth and depth of GPT-4's skills led the authors to state it *"could reasonably be viewed as an early (yet still incomplete) version of an AGI system."* [39] In other words, at sufficiently large scale, an LLM begins to exhibit a general-purpose intelligence, at least in its ability to *output* solutions to diverse tasks.

However, **scale is not a panacea for reliability**. "Reliability" in reasoning means getting consistent, correct results and not being thrown off by traps or corner cases. While larger models are *on average* more accurate and capable, they also demonstrate some new failure modes and inconsistencies:

- **Logical mistakes and hallucinations:** Even GPT-4, for all its power, can **reason incorrectly** at times or assert false facts with confidence. For instance, it might make a subtle error in a multi-step math problem or be fooled by a trick question phrased in an unfamiliar way [40] [41]. As models get larger, their answers sound more fluent and authoritative, which *masks* remaining errors – a reliability

concern. Bigger models are better at *following patterns* but they may also learn very complex spurious correlations that lead to occasional bizarre mistakes (so-called "*Britney Spears is the inventor of COVID*"-style errors, to paraphrase an example from an inverse scaling result).

- **Inverse scaling tasks:** In 2022, a research contest called the **Inverse Scaling Prize** deliberately sought examples of tasks where model performance *worsens* as the model gets larger. Several such tasks were found (often related to prompt trickiness or social bias questions), demonstrating that **bigger models are not *uniformly* more reliable** [42]. As Bengio (2023) highlighted, the existence of inverse-scaling tasks is "alarming" if one assumed larger = more reliable by default [42]. It suggests some forms of reasoning or behavior require more than just scale – they might require explicit training or architectural changes to avoid bad generalizations that scale amplifies.

- **High-stakes reliability and robustness:** Sagawa and colleagues have investigated how *robust* models are as they scale – e.g. do they maintain performance under distribution shifts or adversarial conditions. One finding is a strong correlation between in-distribution and out-of-distribution performance ("Accuracy on the line" – as models get better on their test set, they also *proportionally* get better on OOD data) [43]. That's encouraging (scaling helps robustness broadly), but in high-stakes scenarios (like filtering harmful content, or ensuring no critical errors), just improving average accuracy isn't enough. Techniques like **adversarial training** have been applied to larger models to improve worst-case reliability [44] [45]. Generally, bigger models *need* such mitigations even more, because their mistakes, while rarer, can be more complex and harder to catch (they *almost* get it right, then subtly fail).

So, does scale improve reasoning reliability? **Yes and no.** Yes, in that many reasoning tasks (from symbolic math to multi-hop Q&A) see dramatic jumps in accuracy at certain model sizes – often referred to as **emergent abilities**, where performance transitions from near-random to very strong once a threshold scale is crossed. But also *no*, in that a larger model isn't immune to reasoning traps or systematic biases – it sometimes *amplifies* those as well.

Interestingly, recent work by Ruan *et al.* (2024) offers a nuanced perspective: they argue that what appeared to be "emergent, unpredictable" jumps in ability might actually be **predictable with the right metrics** [46]. By analyzing ~80 different models' capabilities, they found that many complex reasoning tasks follow **smooth sigmoidal scaling curves** (S-shaped) rather than sudden discontinuities [47]. In other words, an ability might look *absent* at small scale and *present* at large scale, but if you plot the performance in between, it's a smooth transition – just that the middle of the "S-curve" was hidden because we didn't have models at those intermediate scales. They showed they could *forecast* the appearance of such abilities by extrapolating from smaller models using a low-dimensional "capability space" representation [46]. For example, they accurately predicted GPT-4's performance on certain "agentic" reasoning tasks by extrapolating from smaller GPT-3 models on easier benchmarks [47]. This is an important insight for **sober forecasting** (discussed more in the next section): it implies that with careful analysis, we might anticipate how reasoning reliability will improve with scale, rather than treating each new model as a total surprise.

Another technique aiding reliability is the use of *prompting methods* like **Chain-of-Thought (CoT)** and **Self-Consistency**. Chain-of-Thought prompting, where the model is guided to produce intermediate reasoning steps, has been found to significantly improve logical and arithmetic problem-solving *especially for larger models* (small models can't really use CoT effectively, but large ones can) [36]. Self-Consistency involves sampling multiple reasoning paths and taking a majority vote on the answer. These approaches effectively

leverage the model's own scale to reduce errors. Crucially, the *benefit* of such methods **scales with model size** – e.g. a 60B model with CoT might outperform a 120B model without CoT on certain tasks, but a 540B model with CoT is better still, and so on [47]. As Ruan *et al.* note, the impact of techniques like CoT can be incorporated into scaling predictions (they showed CoT gives a consistent boost that can be forecasted as models grow) [48] [49]. This means reliability can be partially addressed by algorithmic wrappers around large models, not only by the model's intrinsic weights.

Thought leaders in AI have mixed views on whether scaling alone will eventually produce fully reliable reasoning. DeepMind's Demis Hassabis, for example, has said that *to get all the way to something like AGI, we may require "one or two more breakthroughs" beyond just scaling up existing architectures* [50]. He advocates a hybrid approach: push scaling **and** innovate on algorithms (e.g. incorporating planning, memory, or other reasoning paradigms) in parallel [51]. Yoshua Bengio has expressed even more skepticism that pure scale will make models think like humans, pointing out the need for explicit *inductive biases* for reasoning – architectures that separate "world model" knowledge from "inference" processes, for example [52] [42]. He notes that current large models "do not think like humans do" and continuing to scale them is **unlikely to solve reliability issues by itself** [53] [54]. These experts are essentially cautioning that **quality of reasoning may require qualitative changes**, not just quantitative increases.

On the other hand, the success of GPT-4 and its ilk has made others optimistic that we *might* get quite far just by scaling plus relatively minor fine-tuning. Bubeck *et al.* themselves, after documenting GPT-4's impressive reasoning, still concluded that it is *"still incomplete"* as an AGI and possibly *"we need to move beyond next-word prediction"* to reach deeper reasoning [55]. So even the most scale-enthusiastic research acknowledges something is missing for true robust reasoning – perhaps persistence (the ability to learn continually), or grounding in the physical world, or interpretability to avoid errors.

In practical terms, if you are **planning to scale a model and want it to reason reliably**, you should incorporate measures like: - **Evaluate on reasoning benchmarks across scales:** to detect emergent capabilities or inverse-scaling issues early. - **Use prompting or training tricks** (CoT, etc.) with larger models to enhance their reliability, and understand how those gains scale. - **Allocate part of compute to fine-tuning for reliability** (e.g. instruction tuning, reinforcement learning from human feedback) once the base model is trained. Scaling laws for fine-tuning aren't as well-charted, but as a rule, a larger pretrained model fine-tuned on a given dataset will generally be more data-efficient and performant on that fine-tuning task than a smaller one. - **Monitor worst-case behavior**: As you scale, keep an eye on not just average accuracy but cases where the model completely fails. Sometimes, larger models will have lower overall error rates but might *very rarely* produce a catastrophic error that a smaller model never would (because the smaller model couldn't even attempt the task, for instance). Addressing those might require targeted data augmentation or safety checks.

In summary, scaling definitely improves reasoning ability in many dimensions – GPT-4's near-human performance in diverse fields is a testament to that [38]. But truly **robust, human-level reasoning** might not simply "fall out" of scaling up a Transformer on internet text. It likely will require *engineering* at the data, architecture, or objective level to handle the last-mile reliability issues. From a forecasting standpoint, one should be optimistic about continued capability gains with scale, but cautious about assuming those gains mean the model is **trustworthy and foolproof**. As we scale toward potential AGI, ensuring reliability and alignment becomes as crucial as raw capability.

Sober AGI Timeline Forecasting

When will we have **Artificial General Intelligence (AGI)** – machines that can perform *any* intellectual task a human can, perhaps even improving themselves? This question has moved from speculative to practical as AI capabilities rapidly advance. **Forecasting AGI timelines** means predicting *if* and *when* transformative AI might emerge, using the evidence we have (scaling trends, theoretical arguments, expert opinion). A "sober" approach to this forecasting emphasizes data and cautious reasoning over hype or doomsaying. We will outline key considerations and findings that inform such timelines.

Scaling Extrapolations: One straightforward approach to forecasting is to take the scaling laws and performance curves we've discussed and **extrapolate** them to human-level performance on a broad set of tasks. For instance, if language modeling loss or benchmarks like MMLU have been improving predictably with compute, one could ask: *How much more compute to get to near-100% on these benchmarks?* Kaplan *et al.* in 2020 already speculated along these lines, noting no immediate signs of stopping in the power-law – suggesting **no fundamental barrier in sight** if one could keep increasing compute by orders of magnitude [56]. Back then, one might have estimated (with their exponents) that perhaps a 1000× increase in compute (several more orders of magnitude) could reduce language modeling loss close to the entropy of English – possibly sufficient for very fluent language ability or beyond. Indeed, some in the community voiced a "*scaling hypothesis*" that simply increasing model size and data will eventually yield AGI, as the model's internal representations grow more refined with each increase.

However, **updated scaling laws changed the timeline calculus** somewhat. Using the Hoffmann et al. (Chinchilla) paradigm, if one believes an AGI might require, say, a model with brain-like complexity (often crudely equated to ~10^15 parameters, on the order of synapses in the human brain) or achieving extremely low perplexity, then one must also account for the huge amount of data such a model would need (on the order of 20 × 10^15 ≈ 2e16 tokens). Under Kaplan's old law, one might have trained a 10^15-param model on far fewer tokens and hoped for the best, but Chinchilla's law says you need *proportionally more data* with it [57] [58]. This significantly increases the compute required (because getting 2e16 tokens through the model is enormous). In an analysis by Finnveden (2022), adjusting Ajeya Cotra's influential **"Bio Anchors"** model with Chinchilla's scaling raised the median compute required for a brain-sized model by about **2.5 orders of magnitude** (roughly 300× more) than previously estimated [58] [59]. In effect, if one was anchoring AGI timelines on *simply training a model as big as a human brain*, the **Chinchilla insight pushes that timeline further out**, because it's not enough to build it – you must also train it with unfathomable amounts of data.

On the other hand, if one anchors on *performance trends*, the Chinchilla law can mean **sooner timelines**. Daniel Kokotajlo and others on the Alignment Forum discussed that if models are trained more efficiently, you might reach a given performance threshold with much less total compute than wasteful strategies would have needed [60] [61]. For example, an earlier extrapolation might have said "we need 10^27 FLOPs of training to achieve human-level ability X" but with optimal scaling and smarter training, it might only take 10^25 FLOPs [61]. In fact, when OpenAI released GPT-4 (whose exact training FLOPs are not public, but speculated to be on the order of 10^25 FLOPs), many were surprised how capable it was – some tasks that were expected to need more compute were achieved earlier, likely thanks to efficient use of data and other improvements. So there's a tension: *If our metric is performing well on benchmarks, efficient scaling could deliver that faster.* If our metric is an absolute like "brain-like AGI", our understanding of scaling law nuances could make it seem further.

Expert Surveys and Collective Predictions: Another lens is asking experts when they expect AGI. **Recent surveys show timelines shortening as capabilities advance.** AI Impacts' 2022 and 2023 surveys of hundreds of AI researchers are illustrative. In 2022, the median forecast for a 50% chance of High-Level Machine Intelligence (HLMI, roughly human-job-level AI) was **2060** [62]. Just one year later, after the ChatGPT/GPT-4 leap, the 2023 survey of 2,700 researchers gave a **50% probability by 2047** – a jump of 13 years closer [63] [64]. It also gave a 10% chance of HLMI by 2027 [63], indicating a non-negligible minority think AGI could arrive *within a few years*. This dramatic shift reflects how quickly perceptions change with new evidence: what seemed distant in 2018 or 2020 (when median guesses were late 2050s or beyond [65]) now seems closer. Furthermore, community prediction platforms like Metaculus have also updated sharply. As of early 2024, Metaculus's aggregated forecast for 50% chance of AGI was **2031** [66], which is startlingly soon and about a decade earlier than it predicted a year prior. These "wisdom of crowd" predictions should be taken with caution (they can overreact to recent events), but they underscore that **many informed people now see AGI as quite plausible within the next 10–20 years**.

That said, there's still high uncertainty and variance. Some experts – often those working directly in frontier AI – are more bullish, while others urge caution. It's notable that even within the 2023 survey, definitions and framings affect answers: when asked about *full automation of all jobs*, researchers gave longer timelines (often decades later than for just HLMI) [67] [68]. This shows ambiguity in what "AGI" means to different people. Does it mean human-level on intellectual benchmarks? Replacing humans in essentially all jobs? Something like consciousness or self-improvement? For a **sober timeline analysis**, one should clarify the target. Most current predictions focus on the ability to perform virtually all economically relevant tasks (which is a functional definition of transformative AI).

Public statements: The leaders of AI labs sometimes share their views. Demis Hassabis (DeepMind) in May 2025 opined that we will get to AGI with a combination of scale and new ideas, and he did **not** flatly say "in 2 years" or such – he indicated a couple more breakthroughs might be needed [50]. Sam Altman (OpenAI) has hinted he doesn't believe we already have AGI but that it could be within a decade or so, given the exponential progress – though he's also said it's hard to predict and might take longer. On the more optimistic end, some tech executives (like a leaked Google memo or certain VC investors) have claimed it could be extremely soon (a few even say *it's already sort-of here* with GPT-4). On the skeptical end, scientists like Yann LeCun or Gary Marcus argue that current LLMs are *not the right path* to AGI at all, and without new paradigms (e.g. LeCun's views on architectures with explicit world models and planners) we won't reach human-level AI by just scaling GPT-like models. These disagreements highlight that **forecasting is inherently uncertain** – even experts disagree both on timeline and on *methodology* (scale vs algorithm).

The importance of being "sober": In this context, a sober forecast means acknowledging uncertainties and avoiding both unjustified optimism and pessimism. It means using data like scaling trends and current capabilities as a guide, but also recognizing the potential for diminishing returns or unexpected hurdles. Here are a few sober assessments one might make:

- **Computational and economic limits:** Even if scaling laws suggest AGI is reachable with, say, 1000× more compute than GPT-4, will we actually deploy such compute? The cost might be prohibitive or require new hardware advances. If Moore's Law or its equivalent slows, timelines could extend. On the flip side, if there are major breakthroughs in hardware (quantum computing, optical chips) or algorithm efficiency, timelines could accelerate.

- **Algorithmic breakthroughs:** A breakthrough (like a new architecture or learning method) could dramatically reduce the compute needed for a given level of intelligence – which would make AGI arrive faster than scaling curves alone predict. For instance, if someone figures out how to do unsupervised learning of world models in a way that reduces required data by 100×, that changes things. Many expect that at least *some* new ideas (one or two "missing pieces") are needed for true AGI [50], but it's hard to predict *when* those will come (tomorrow or in 10 years?).

- **Alignment and safety factors:** A model might be capable of AGI-level performance but we might intentionally hold it back or not deploy it until we are confident in its alignment (safety). If major AI labs become cautious (as some did after GPT-4, taking longer to release GPT-4 fully), this could delay when the world actually *sees* an AGI in practice, even if technically feasible.

Combining these, a reasonable middle-of-the-road prediction (as of 2025) might be: *By the mid-2030s*, we have a decent chance of AI systems that can outperform humans on nearly all cognitive tasks (what some call HLMI or even "weak AGI"). The 2040s seem quite likely for that level if progress continues, given the survey trends [63] [69]. However, achieving *full* human-level generality – including things like adaptive learning, self-directed goal-setting, common sense grounding in the physical world, etc. – might stretch longer, especially if new approaches are needed beyond scaling LLMs. Many experts put non-trivial probability on *never* reaching a human-like AGI (because perhaps it needs fundamentally different paradigms), but the majority now appear to assign >50% odds that it will happen *this century*, and a significant chunk think **within the next 20 years** [63] [70]. These are sobering in that they're neither "this will solve everything in 2 years" nor "hundreds of years away" – they suggest we should prepare for the real possibility of AGI in our lifetime, while also acknowledging we could be wrong in either direction.

To ground this in our earlier topics: scaling laws give one lens on timelines. They say: *if we continue the exponential growth in compute and data, we will continue to get exponential (or faster) improvements in capability – until we hit some fundamental limit.* So far, we haven't hit a hard limit, though data scarcity and costs are starting to bite. Forecasting based on those, and observing the acceleration from GPT-3 (2020) → GPT-4 (2023), it's not unreasonable to suspect another few iterations might get us extremely far. But as a sober forecaster, one should also plan for scenarios where progress slows: perhaps scaling further yields diminishing returns (as tasks get harder, like truly understanding complex real-world concepts, the power-law might bend or require a qualitatively new type of data, e.g. video or interaction). The *observational scaling* work by Ruan et al. suggests we can track many capabilities as smooth functions of scale [46]; if we start to see those curves flatten, that's evidence AGI might not just pop out without new ideas. If they keep rising steadily, then each year of scaling (assuming budgets allow) brings us closer on all fronts.

In conclusion, **AGI timeline forecasting** has evolved from a hypothetical musing to a data-informed projection in which scaling plays a starring role. We leverage scaling laws, like those by Kaplan and Hoffmann, to estimate the compute needed, and we incorporate the latest results (GPT-4's "sparks", etc.) as evidence of how close we are. The consensus is shifting to earlier timelines but with significant error bars. The prudent approach is to *prepare for sooner while not assuming it's a solved path*. As we plan actual scaling projects, we do so with an eye on these forecasts, because the *context* in which we scale a model today is that it might be part of a trajectory leading to AGI in the not-too-distant future.

Case Study: Crafting a Scaling Plan (Group Proposal Guide)

To solidify understanding, let's walk through how one might create a **scaling plan** for a hypothetical project, integrating the ideas above. Imagine your team is tasked with building a model that can reliably perform a suite of complex reasoning tasks (say, pass a difficult professional exam, write robust code, and carry on expert-level dialogues) – essentially a mini-AGI for a specific domain. You have certain compute resources available. How do you plan the scaling strategy? Below is a structured approach:

1. Set Clear Performance Goals: First, define what success looks like. Is it achieving human-level score on a specific benchmark (e.g. >80% on an exam)? Solving 90% of coding problems from a set? This goal will inform how *far* along the scaling curve you need to get. For instance, if current state-of-the-art (say GPT-3 level) is at 50% and you need 80%, that gap gives a clue of how many orders of magnitude improvement might be needed (based on past scaling trends, maybe one or two orders of magnitude more compute). Also decide any reliability targets (e.g. model should make less than 1% critical errors). Being explicit helps in planning data and evaluation.

2. Compute Budget and Model Sizing: Determine your training compute budget – e.g. "We have 1e24 FLOPs available for training this model" (which is on the order used for some recent large models). Using **compute-optimal scaling laws**, allocate this between model size and total tokens. Following Hoffmann et al.'s rule, *plan to use roughly 50% of compute on increasing model size and 50% on increasing training length*. For example, with 1e24 FLOPs, you might choose a model with ~50B parameters and plan to train on ~1e12 to 2e12 tokens (just illustrating). You can use formulas: $N \propto C^{0.5}, D \propto C^{0.5}$ [14]. A quick method is to start from a known reference: If a 10B model on 200B tokens used 1e22 FLOPs (roughly), then 1e24 FLOPs is 100× more, whose square root is 10× for model and data each. So you'd target ~100B params and ~2 trillion tokens to use the 100× compute efficiently. Since you might not have exactly the "Chinchilla-optimal" amount of high-quality data, you could adjust slightly (e.g. if only 1e12 tokens available, you might reduce model size a bit so as not to undertrain). The key is to **stay near the efficient frontier** – don't, say, spend 90% of FLOPs on a giant model if you can't afford the tokens to train it well, and vice versa.

3. Data Acquisition and Quality Plan: Outline the datasets you will use and how you will ensure quality. Following the discussion above, list sources of high-quality data relevant to your tasks (e.g. a large corpus of textbooks and solutions if you're aiming at exam problems, high-quality code repositories for coding tasks, etc.). If existing data is limited, include a strategy for **data augmentation**: perhaps use a smaller model to generate additional training data (like phi-1 did with GPT-3.5 creating synthetic exercises) [31]. Plan for data filtering – you might decide to filter out content below a certain quality score or remove duplicates to reduce redundancy (which contributes to sub-scaling [35]). If your target tasks require specialized knowledge, ensure your training data covers that (even if it means web scraping academic articles or fine-tuning on domain-specific data later). Essentially, the plan should ensure that **data won't be your bottleneck** in training – both in quantity and in richness. It's often wise to prepare a *curriculum*: e.g. start training on a broad base of simpler texts and gradually up-weight harder, more domain-specific data as the model learns (some evidence suggests models learn better that way, and it ensures later training focuses on the high-value data once the model is ready to absorb it).

4. Training Strategy and Schedule: Lay out how the training will proceed. Specify hyperparameters guided by scaling law insights (e.g. appropriate learning rate schedule for a long training on lots of data – Hoffmann et al. noted that too-short cycles can undertrain a large model [71], so you may use a cosine decay over the full run, etc.). Plan for *early stopping criteria* – since compute-optimal implies stopping before

full convergence [8], decide how you'll determine when to stop (maybe when validation loss flattens to the irreducible level, or when further improvement would yield tiny returns compared to cost). Also, consider doing intermediate model evaluations at various scales: for example, you could *train smaller proxy models (like 1B, 10B)* on subsets of data first to verify scaling trends and adjust your approach (this is akin to the *"observational scaling"* idea: use many small models to predict big model behavior [72] [46]). Your plan might include a *phase 1* where you train a series of models at 1B, 5B, 10B parameters on maybe 100B tokens to see how loss is trending and whether the empirical scaling matches expectations. This can inform if your data is as useful as you think – if not, you might pivot to improve data before training the full 50B model.

5. Incorporate Reliability Measures: Given the importance of reliability in reasoning, build in steps to make the model's output more trustworthy: - Plan a **fine-tuning stage**: After the main pretraining on generic data, allocate some compute for fine-tuning on specific tasks or with human feedback. For example, do an instruction-tuning pass where the model learns to follow human instructions and produce reasoning chains. Or use Reinforcement Learning from Human Feedback (RLHF) to align it with desired behaviors (this is how models like ChatGPT/GPT-4 improve factuality and avoid egregious errors). This fine-tuning can dramatically increase reliability on the kinds of queries it will get, even if it's not part of the initial scaling law per se. - Plan to use **Chain-of-Thought prompting** during deployment or as part of training. For instance, you might fine-tune the model to generate step-by-step solutions for problems (thereby teaching it to actually "think out loud"). If reliability in complex reasoning is paramount, you could also implement *self-consistency decoding*: generate multiple solutions and have the model or a simple algorithm vote on the answer [48]. Your scaling plan can mention that you will evaluate the impact of CoT on smaller models and ensure the large model is compatible with it. - **Adversarial testing:** Include a stage in the plan where you will test the model on adversarial or "edge" cases (perhaps using techniques from the Inverse Scaling Prize or adversarial training literature [44]). If the model shows particular weaknesses (say, it falls for a trick question consistently, or it hallucinates sources in a knowledge query), you can then address this by either adding training data for those cases or adjusting the decoding method. Essentially, don't assume the scaled model will be perfectly reliable – build in a feedback loop to catch and correct failure modes.

6. Forecast Expected Performance: Using scaling laws and any pilot experiments, predict what performance the final model is likely to achieve. For example: *"Based on the scaling law exponent observed, a 10× increase in model size and data gave a ~X% reduction in error; therefore, our 50B model (5× bigger than baseline) on 10× data is expected to reach ~Y accuracy on the validation tasks."* Make use of the empirical data: if smaller models follow a power-law, extrapolate that (maybe even fit your own scaling curve). Also consider the **sigmoidal emergence** pattern – if a key ability is expected to "emerge" around 20B, our 50B model will be safely on the plateau of that S-curve, so it should solidly possess that ability. For instance, if at 6B parameters the model started to solve 2-digit multiplication and by 52B it's near 100% on it, then 50B will definitely handle it. Listing these expectations helps when you later evaluate the model: if it falls short, that indicates either a data issue or a deviation from the prior trends.

7. Risk Mitigation and Contingency: A good plan will also mention what you'll do if things don't go as expected. If halfway through training the loss is not decreasing as it should (perhaps indicating a **data quality problem or a optimization issue**), how will you adjust? You might set a milestone: "If at 20% of training, loss > target by X, then pause and analyze – maybe increase learning rate, or filter data more." Consider the possibility of hitting a bottleneck: e.g., *data runs out* – then maybe you would fall back to reusing data with augmentation (acknowledging some sub-scaling might occur). Or if the model is great on average but *still makes specific reasoning errors*, have a plan to address those (maybe a specialized module

or post-processor can catch them, or further fine-tuning). Essentially, show that the plan isn't blindly assuming the scaling law will do everything – you have checkpoints to verify and course-correct.

8. Deployment and Inference Considerations: (Optional, but important in reality) – If this model will be deployed for interactive use, consider the *Chinchilla vs. inference trade-off*. For example, **Sardana et al. (2023)** pointed out that a strictly compute-optimal (Chinchilla) model might be very large and thus expensive to *use* in practice, and that if you expect heavy usage, a slightly smaller model trained longer could be more cost-effective overall [73] [74]. In a proposal, you might note: "Our chosen model size of 50B is partly to ensure inference is feasible within latency limits, whereas a 200B model would have been too slow/expensive to serve – we compensate by training longer on more data to reach the needed performance" [73] [74]. This shows awareness of real-world constraints: scaling isn't just about training, but also about the end-use of the model.

By following these steps, your group's scaling-plan proposal would demonstrate a comprehensive understanding of scaling strategy. You'd cite Kaplan et al. (2020) for initial expectations, Hoffmann et al. (2022) for the crucial model-data balance [13], and perhaps refer to cases like phi-1 for data quality emphasis [31]. You'd mention Bubeck et al. (2023) as motivation that reaching certain scales can unlock new capabilities [38], but also Bengio or others to justify why you include special measures for reliability [42]. The plan would effectively be a *case study* in applying all these lessons: use scaling laws as a guide, but also address the practical realities of data and reliability, and keep an eye on the ultimate goal of robust capability.

Conclusion: Scaling laws and strategies form the roadmap by which we navigate toward more powerful AI systems, and forecasting capability frontiers helps us anticipate what lies over the horizon. We have learned that scaling is a highly effective tool – models have consistently gotten better with more parameters, more data, and more compute – but using it optimally requires insight (as the shift from Kaplan's to Hoffmann's scaling law taught us [13] [23]). We must also mind the quality of the journey: ensuring our data is useful and our models remain reliable as they grow. By combining empirical scaling trends with careful planning and a dose of caution, we can chart a course that maximizes our chances of success in building the next generation of intelligent, trustworthy models. The study of scaling is not just about making models bigger – it's about **making them better in a predictable, efficient way**, and being prepared for the transformative potential they bring. With this knowledge, your group is well-equipped to propose a scaling strategy that is ambitious, evidence-based, and responsibly mindful of the challenges and opportunities on the path to AGI.

Sources: Kaplan et al., 2020 [2] [5]; Hoffmann et al., 2022 [13] [23]; Chen et al., 2025 [27] [26]; Gunasekar et al., 2023 [31]; Bubeck et al., 2023 [38] [39]; Bengio, 2023 [42]; AI Impacts Survey 2023 [63] [69]; Alignment Forum discussions [57] [58]; and others as cited throughout.

---

1 4 5 6 7 9 10 14 15 16 17 18 19 20 21 22 33 56 71 73 74 Scaling Laws for LLM Pretraining
https://www.jonvet.com/blog/llm-scaling-laws

2 3 8 [2001.08361] Scaling Laws for Neural Language Models
https://arxiv.org/abs/2001.08361

11 12 13 23 24 [2203.15556] Training Compute-Optimal Large Language Models
https://arxiv.org/abs/2203.15556

25 26 27 28 29 30 34 35 aclanthology.org
https://aclanthology.org/2025.acl-long.1163.pdf

31 [2306.11644] Textbooks Are All You Need
https://arxiv.org/abs/2306.11644

32 Assessing Data Quality through Inverse Utilization of Scaling Laws
https://arxiv.org/html/2408.08310v1

36 37 40 41 Beyond Accuracy: Evaluating the Reasoning Behavior of Large Language Models - A Survey
https://arxiv.org/html/2404.01869v1

38 39 55 [2303.12712] Sparks of Artificial General Intelligence: Early experiments with GPT-4
https://arxiv.org/abs/2303.12712

42 52 53 54 Scaling in the service of reasoning & model-based ML - Yoshua Bengio
https://yoshuabengio.org/2023/03/21/scaling-in-the-service-of-reasoning-model-based-ml/

43 46 47 48 49 72 Observational Scaling Laws and the Predictability of Language Model Performance
https://arxiv.org/html/2405.10938v1

44 45 [2205.01663] Adversarial Training for High-Stakes Reliability
https://arxiv.org/abs/2205.01663

50 51 Demis Hassabis and Sergey Brin on AI Scaling, AGI Timeline, Robotics, Simulation Theory
https://www.bigtechnology.com/p/demis-hassabis-and-sergey-brin-on

57 58 59 60 61 PaLM in "Extrapolating GPT-N performance" — AI Alignment Forum
https://www.alignmentforum.org/posts/YzbQeCiwoLBHrvAh4/palm-in-extrapolating-gpt-n-performance

62 63 64 65 66 67 68 69 70 Timelines to Transformative AI: an investigation — EA Forum
https://forum.effectivealtruism.org/posts/hzhGL7tb56hG5pRXY/timelines-to-transformative-ai-an-investigation